

LL as Ket: Scope Selection and the Folded 0xE_ Manifold

0. Purpose

This document defines the role of `LL` inside the OMI truth row.

The truth row is:

```
(LL, NN, MM)
```

Earlier descriptions treated `LL` mainly as a lens or modulator. That remains correct, but the completed `0xE_` execution band gives `LL` a sharper role:

```
LL is the ket-like scope selector of the frame.
```

It does not contain the object.

It does not contain geometry.

It does not contain application policy.

It selects how a proven transition lands inside the accepted execution manifold.

In OMI terms:

```
LL scopes.  
NN begins.  
MM resolves.  
Delta proves.  
Fano bounds.  
Receipt authorizes.  
Rule block materializes.  
Projection displays.
```

1. The Truth Row

Every accepted OMI frame yields one truth row:

(LL, NN, MM)

Where:

LL = scope / lens / ket selector
NN = antecedent state
MM = consequent state

The transition being proven is:

NN -> MM

That transition is not accepted because it renders, parses, or appears in a portal.

It is accepted only when:

Q_frame(S) = 0
Delta/Fano resolves NN -> MM
receipt records the accepted envelope

After that, LL determines where the accepted transition materializes.

Short form:

NN/MM prove the transition.
LL routes the accepted transition.

2. LL as Ket

In quantum notation, a ket such as:

$|\psi\rangle$

represents a state vector.

OMI does not need to import quantum mechanics literally, but the analogy is useful.

LL behaves like a small bounded ket:

```
|LL>
```

It is the selector state that scopes how the frame is encountered by the materializer.

The OMI truth row can therefore be read as:

```
(|LL>, NN, MM)
```

or:

```
scope ket + antecedent + consequent
```

`LL` is not the whole state.

`LL` is not the object body.

`LL` is not the semantic meaning.

`LL` is the scoped selector that tells the accepted frame where to land.

3. The Nibble Split

`LL` is interpreted as two nibbles:

```
LL = 0xAB
```

Where:

```
A = high nibble
```

```
B = low nibble
```

OMI reads them as:

```
high nibble = context / lens family / carry-forward scope  
low nibble  = execution slot selector
```

The POSIX gate computes:

```
slot_index=$((ll & 0x0f))
rule_slot=$((0xe0 + slot_index))
context=$((ll >> 4) & 0x0f))
```

This gives the execution envelope a clean orthogonal structure:

```
context    = high-nibble scope
slot_index = low-nibble selector
rule_slot  = 0xE0 + slot_index
```

The shell does not branch on meaning.

It only extracts structure.

4. The Folded 0xE_ Manifold

The low nibble of `LL` addresses the complete `0xE_` execution band:

```
0xE0-0xEB = twelve inner materializers
0xEC-0xEF = four outer controller vertices
```

The old `LL mod 12` reading was useful for the inner kissing-sphere metaphor:

```
0xE0-0xEB = 12 packed inner operations
```

But the completed execution manifold requires the full nibble:

```
LL & 0x0F
```

This saturates all sixteen slots:

```
0x0 -> 0xE0
0x1 -> 0xE1
0x2 -> 0xE2
0x3 -> 0xE3
0x4 -> 0xE4
0x5 -> 0xE5
0x6 -> 0xE6
0x7 -> 0xE7
```

```
0x8 -> 0xE8
0x9 -> 0xE9
0xA -> 0xEA
0xB -> 0xEB
0xC -> 0xEC
0xD -> 0xED
0xE -> 0xEE
0xF -> 0xEF
```

So the final routing law is:

```
rule_slot = 0xE0 + (LL & 0x0F)
```

This is the closure of the 0xE_ band.

5. The First Twelve and the Last Four

The 0xE_ band is one accepted execution manifold, but it contains two regions.

5.1 Inner Materializer Field

```
0xE0-0xEB
```

These twelve slots are the inner packed materializers.

They correspond to the Archimedean/Catalan materializer field: the place where accepted object transformations, snub bridges, edge traversal, vertex traversal, Fano selection, page framing, DOM/CSSOM/JSDOM/OOM patching, and receipt projection may be materialized by registry-defined operations.

They pack the object.

```
0xE0-0xEB pack the object.
```

5.2 Outer Controller Vertices

```
0xEC-0xEF
```

These four slots are the outer Platonic controller vertices.

They do not form a separate authority channel. They are not UI commands floating outside the protocol. They are the last four addresses of the same accepted execution manifold.

They hold the object.

0xEC-0xEF hold the object.

They may control:

0xEC = translation / viewport / focus
0xED = scale / recursion depth
0xEE = rotation / spin / orientation
0xEF = boundary / opacity / clipping

But only after the frame has been validated, resolved, and receipted.

6. The Platonic Fold

The final four controller slots can be read as Platonic controller vertices folded into the 0xE_ band.

They are not extra commands.

They are the tetrahedral control surface of the execution manifold.

In geometric language:

0xE0-0xEB = Archimedean/Catalan materializer field
0xEC-0xEF = Platonic controller vertices

The first twelve expose the packed operational field.

The last four close the outer simplex.

This means the control surface is not outside the system. It is folded into the same accepted manifold as the object operations.

Short form:

The first twelve pack.
The last four hold.

The low nibble selects.
The high nibble carries.

7. Carry-Forward Scope

The high nibble of `LL` carries context forward.

```
context = (LL >> 4) & 0x0F
```

This can be understood as a 4-bit carry-forward scope.

It does not decide semantic meaning inside the shell. It gives the materializer a bounded context value.

A materializer may interpret context as:

```
lens family
authority context
projection family
object scope
depth band
peer scope
document scope
portal scope
```

But the gate does not interpret it.

The gate only emits it.

If the current scope is represented as a 4-bit value, the next scope may be derived by XOR:

```
next_scope = current_scope xor context
```

This gives OMI a bounded carry-forward model:

```
scope' = scope ⊕ high_nibble(LL)
```

The XOR model is useful because it keeps the scope update bitwise, bounded, reversible in form, and compatible with the OMI preference for low-level operations.

8. Low Nibble as Slot Selector

The low nibble of `LL` selects the rule slot:

```
slot_index = LL & 0x0F  
rule_slot  = 0xE0 + slot_index
```

The low nibble does not contain geometry.

It only selects an address.

The registry owns meaning.

For example:

```
LL = 0x07  
context = 0x0  
slot_index = 0x7  
rule_slot = 0xE7
```

The registry may define:

```
0xE7 = snub-bridge
```

Another example:

```
LL = 0x3E  
context = 0x3  
slot_index = 0xE  
rule_slot = 0xEE
```

The registry may define:

```
0xEE = rotation-spin
```

The shell does not know either name.

It only knows the frame was accepted and the envelope was recorded.

9. NN and MM as the Proven Pair

NN and **MM** remain the transition pair.

NN = antecedent
MM = consequent

The resolver proves that **MM** is reachable from **NN** under the accepted Delta/Fano path.

The **LL** ket does not replace this proof.

It scopes it.

So the relation is:

Delta/Fano proves NN -> MM.
LL selects where the proof lands.
Receipt records the accepted envelope.

This prevents the selector from becoming authority by itself.

LL cannot accept a transition.
LL cannot bypass **Q_frame**.
LL cannot bypass Delta/Fano.
LL cannot bypass receipt.

It only routes the accepted transition.

10. Execution Envelope

The accepted frame emits an envelope such as:

```
{
  "accepted": true,
  "LL": "0x3E",
  "context": "0x3",
  "slot_index": "0xE",
  "rule_slot": "0xEE",
  "rule_band": "0xE0-0xEF",
  "NN": "0x7c00",
  "MM": "0x1434",
  "steps": 3,
```

```
"chirality": "right",  
"receipt": "..."  
}
```

The materializer receives:

```
context = 0x3  
rule_slot = 0xEE  
steps = 3  
chirality = right  
receipt = accepted
```

Then it applies the registered operation for `0xEE`.

The materializer may know that `0xEE` means rotation/spin.

The gate does not.

11. Steps and Chirality

The execution envelope gives the materializer four orthogonal parameters:

```
context  
rule_slot  
steps  
chirality
```

Their roles are:

```
context    = lens family / carry-forward scope  
rule_slot  = operation address  
steps      = magnitude / phase / recursion depth  
chirality  = handed direction
```

Example:

```
rule_slot = 0xEE  
context = 0x3  
steps = 3  
chirality = right
```

The materializer may interpret this as:

```
rotate the projection under context 0x3 by three accepted steps in the right-  
handed direction
```

But this is materialization policy, not gate policy.

12. No Separate Controller Channel

The controller vertices `0xEC-0xEF` must not exist on a separate command channel.

A separate channel would create double authority.

It would allow a projection or UI to move the lens outside the receipt path.

OMI rejects that.

The rule is:

```
No controller outside receipt.
```

A viewport, scale, rotation, boundary, opacity, or clipping change may be local and non-authoritative. But if it claims to be an accepted OMI view operation, it must come through the same frame pipeline:

```
validate -> resolve -> record -> materialize -> project -> inspect
```

The stricter viewport language is:

```
The viewport is receipt-bound when controlled through 0xEC-0xEF. A conforming  
projection must not treat out-of-band camera or lens changes as accepted state.
```

This preserves the separation:

```
local display may vary  
accepted projection must be receipted
```

13. Gate and Materializer

The gate is `start.sh`.

The materializer is the rule registry and its implementations.

The gate does:

```
parse frame
validate Q_frame(S) = 0
extract (LL, NN, MM)
resolve NN -> MM
derive context
derive slot_index
derive rule_slot
write receipt
emit envelope
```

The gate does not:

```
know snub geometry
know Platonic solids
know Archimedean/Catalan duals
know DOM policy
know CSSOM policy
know JSDOM policy
know A-Frame policy
know portal UX
```

The materializer does:

```
read accepted envelope
lookup rule_slot
apply registered pure implementation
respect context
respect steps
respect chirality
project result
explain result
```

The split is absolute:

```
start.sh is the gate.  
The registry is the materializer.
```

14. Relation to the 4D Simplex

The final four controller vertices can be read as a 4-simplex-style closure surface.

They are the folded controller vertices that make the materializer manifold complete.

The first twelve slots give the dense packed field.

The last four slots give the outer controller surface.

This means the OMI execution band is not merely a list of sixteen commands. It is a folded topology:

```
twelve inner operations  
four outer vertices  
one accepted manifold
```

The `LL` ket selects into that manifold.

The `NN/MM` pair proves the transition.

The receipt closes the event.

15. Final Canon

The OMI truth row is:

```
(LL, NN, MM)
```

`NN` and `MM` are the proven transition pair.

`LL` is the ket-like scope selector.

Its high nibble carries forward context into the next scope.

Its low nibble selects one address in the folded `0xE_` execution manifold.

The first twelve addresses expose the Archimedean/Catalan materializer field.

The final four addresses expose the Platonic controller vertices of the simplex closure.

The gate remains blind.

The registry owns meaning.

Short doctrine:

```
LL scopes.  
NN begins.  
MM resolves.  
Delta proves.  
Fano bounds.  
The high nibble carries.  
The low nibble selects.  
The first twelve pack.  
The last four hold.  
Receipt closes.
```

One-line canon:

```
LL is the ket of the frame: a 4-bit carry-forward scope selector whose low  
nibble lands the accepted NN->MM transition inside the folded 0xE_ materializer  
manifold.
```